Credit- and Deadline-Aware Flow Control for a Shallow-Buffer Die-to-Die Link under Tiled General Matrix Multiply Traffic

Die-to-die hop architecture

The paper is one hop and one score block. The architecture below is that hop. The strategy at the end is how the paper is completed. The link is die-to-die (D2D). The traffic is general matrix multiply (GEMM). The design is register-transfer level (RTL). The score has five fills. Two of them are first-come, first-served (FCFS) and earliest deadline first (EDF). The other three are shortest remaining, most consumers, and longest remaining. The hop delay is the physical layer (PHY). Other flits are illustrated by Compute Express Link (CXL).

1 Purpose

The paper studies one D2D hop with one score block. The data die sends the A panel and the B panel of each output tile. The compute die forms C. The bytes of C stay on the compute die. `POLICY`, the build parameter of the score block, is chosen when the chip is built. It packs selected fields of the ready record into one 48-bit score. The scheduler compares that score. A smaller score wins.

The objective is to choose the fill that keeps GEMM output tiles inside D_k, the deadline of output tile k, when other flits share the same K receive slots. The hard part is the shallow receive side. An other flit can take a slot that an output tile still needs.

2 Set up

2.1 Context

Two dies share one hop. One output tile k carries panel A and panel B for that tile. It does not carry the whole of A. It does not carry the whole of B. `panel_sel`, the panel bit, is 0 on an A flit and 1 on a B flit. Every flit of the tile keeps `tile_id`, the index of that output tile. The deadline stamped on the data die is $D_k = 16k + 40$.

The compute die holds at most K flits. K counts receive slots. A send from the data die spends one credit. That credit returns after the flit has occupied a slot on the compute die.

2.2 Panels

One flit is 32 bytes, which is 256 bits.

- Panel A, on the data die, is 256 bytes. That is 8 flits.
- Panel B, on the data die, is 256 bytes. That is 8 flits.
- One output tile, A then B, is 512 bytes. That is 16 flits.

2.3 Problem

K receive slots are shared by the GEMM flits and by other flits. A CXL stream is the example of those other flits. A send that follows packing order can give a slot to an other flit while output tile k is still waiting. That tile can then arrive after D_k.

3 Proposed policies

The work is one hop, with one score block. An empty record cannot win. An equal score keeps the left input. On this tree the left input is the lower slot. The control unit has no policy pin. The scheduler has no mux in front of it.

The five packings are five fills of that same score word. A rebuild selects one of them.

- 0, FCFS. The packing order `seq`.
- 1, EDF. D_k, then `seq`.
- 2, shortest remaining. `flits_left`, then `seq`. Fewer flits left wins.
- 3, most consumers. `panel_sel`, then the inverted `consumers` count, then `flits_left`, then `seq`. Panel A ranks ahead of panel B. A larger consumer count ranks ahead of a smaller one.
- 4, longest remaining. The inverted `flits_left`, then `seq`. More flits left wins.

The credit gate is the shallow buffer. A flit leaves the data die only while the compute die has a free slot. The PHY delays the record by D_{phy} cycles. It does not read the score. On the compute die, unpack returns one panel byte with the tile and the panel that travelled with it.

4 Workflow

The flow cites the requirements in the datasheet. It does not restate them. Admission is R1 and R2. Packing and the lane close are R3 and R4. The deadline stamp is R5. The score fill is R6. The lowest score is R7. One offer of a ready record is R8. Credit safety and credit return are R9 and R10. Tile completion is R11. Delivery is R12.

```
hop_path
  u_cu -> release_en, W, admit_limit, send_en
  u_release -> u_packer -> u_ready -> u_score
  u_ready -> u_sched -> u_credit -> u_phy -> u_unpack
```

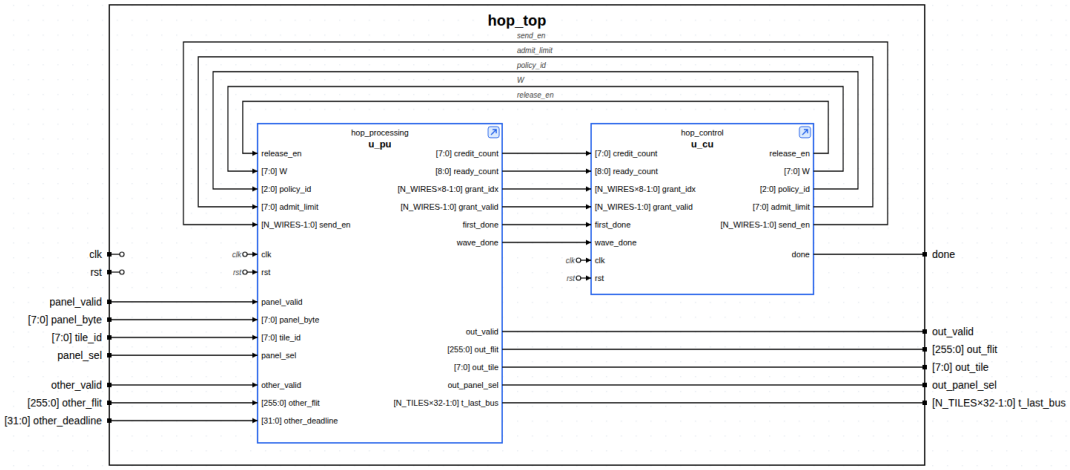


Figure 1: Editor sheet of the top view. The frame is `hop_top`. The blue frames are `u_pu` (`hop_processing`) and `u_cu` (`hop_control`).

A dashed stub marked “no driver” means that pin has no source on the sheet. No block in that view writes the net.

On the processing sheet, `square_finished`, `credit_return`, and `t_last_bus` carry that mark.

`first_done` and `wave_done` carry it too. In `hop_path`, `u_ready` writes `square_finished` when the 16th GEMM flit of a tile leaves. `u_credit` writes `credit_return` from the last stage of its delay line. `first_done`, `wave_done`, and `t_last_bus` are not pins of `hop_path`.

The unpack sheet marks `out_tile` and `out_panel_sel` the same way. `u_phy` drives both from the delayed record.

A wave is the run of output tiles admitted under one held length. On the first output tile, `u_release` copies W into `w_hold`. The wave ends when that many output tiles have been admitted. Each output tile keeps its own D_k. Output tiles enter one at a time, when the cap has room and the byte lane is idle.

`u_cu` sets release and send permission. It does not carry panel bytes. `POLICY` is chosen at build time. The compare finishes in the cycle its inputs arrive. The granted record then waits on a credit. `pass_en` loads the PHY and frees the ready slot. `out_valid` is D_{phy} cycles later. The credit returns $2D_{\text{phy}}$ cycles after the same send.

The four PHY registers sit in the digital adapter. Each clock moves the record one stage. The package trace is outside this RTL. The registers are the cycle count of that flight. A quiet cycle shifts a bubble.

5 Testbench

The bench has 22 cases in three groups. Nine cases drive one pair. Ten cases drive the four-leaf tree. Three cases drive a broken compare that must fail. The compare starts from a score that is already packed. It does not pack panel A or panel B. The five fills are requirement R6.

Case	Group	What it drives	Expected	Result
1	pair	Both inputs invalid.	Valid 0. Index 7.	Pass
2	pair	Only the right input is valid.	Valid 1. Index 9.	Pass
3	pair	Only the left input is valid.	Valid 1. Index 7.	Pass
4	pair	Right score is smaller.	Valid 1. Index 9.	Pass
5	pair	Left score is smaller.	Valid 1. Index 7.	Pass
6	pair	Equal scores.	Valid 1. Index 7.	Pass
7	pair	The scores differ by one.	The smaller score wins.	Pass
8	pair	One score is zero. The other has only its highest place set.	The smaller score wins.	Pass
9	pair	The index rides with the score.	Output index is the winner's index.	Pass
10	tree	All four valid bits clear.	Grant valid is 0.	Pass
11	tree	Masks 0 through 15. Scores 16, 4, 12, and 8.	Lowest valid score. Slot 1 can beat slot 0.	Pass
12	tree	Only slot 2 is valid.	Grant is slot 2.	Pass
13	tree	Score 1 moves through each slot.	Grant follows that slot.	Pass
14	tree	Tie on slots 1 and 3.	Lower slot wins. Index 1.	Pass
15	tree	Ties inside each pair.	Lower slot of the pair wins.	Pass
16	tree	All four scores equal.	Index 0.	Pass
17	tree	High half $2^{32} - 1$ against high half 1.	The smaller high half wins. Index 3.	Pass
18	tree	1000 random scores.	Each grant matches the tree rule.	Pass

Case	Group	What it drives	Expected	Result
19	tree	1000 scores drawn from 0 to 3.	Ties keep the lower slot. 210 ties.	Pass
20	mutant	Compare uses $\leq$.	Tie takes the right side.	Fail
21	mutant	An invalid leaf is not forced to lose.	A small score on an invalid leaf can win.	Fail
22	mutant	Left pair slot ids are swapped.	Reported index is the other slot.	Fail

6 Datasheet

6.1 Requirement

Twelve requirements name the block that meets them. The pair and tree bench covers R7. R6 is the score fill. The hop path covers admission, packing, the wave hold, credit, and delivery.

R1. Bounded admission. At most $\min(W, \texttt{admit_limit})$ output tiles are inside the hop, using the live W. A new tile is admitted only when `release_en` is high, the count is below that cap, and the byte lane is idle. *Block:* `u_release`. *Check:* the admitted count never exceeds the cap.

R2. Wave length. On the first output tile of a wave, W is copied into `w_hold`. The wave ends when that many output tiles have been admitted, even if the live W later falls. *Block:* `u_release`. *Check:* a wave started at 4 still ends on its fourth output tile.

R3. Lossless packing. Every 32 accepted bytes form one flit. One output tile gives 16 flits carrying its `tile_id`, `panel_sel`, and wave id. *Block:* `u_packer`. *Check:* the unpacked bytes match the bytes that were accepted.

R4. Lane close. `pack_done` pulses on the 16th packed flit and only closes the byte lane. The output tile stays admitted. *Block:* `u_packer`, `u_release`. *Check:* the admitted count is unchanged on `pack_done`.

R5. Deadline stamping. Every GEMM record stores $D_k = 16 \cdot \texttt{tile_id} + 40$. Every other-stack record stores the deadline it brings. *Block:* `u_ready`. *Check:* the stored deadline equals that value.

R6. One score packing. The hop is built with one `POLICY`. That parameter fills one 48-bit score. There is no policy pin and no second hop. For tile 3, sequence 0, panel A, $D_k = 88$, $\texttt{flits_left} = 16$, and $\texttt{consumers} = 1$, the fill is one of these.

- FCFS. Sequence 0.
- EDF. 88, then sequence 0.
- Shortest remaining. 16, then sequence 0.
- Most consumers. Panel 0, inverted consumers 254, then 16, then sequence 0.
- Longest remaining. Inverted `flits_left` 15, then sequence 0.

Block: `u_score`. *Check:* the packed word of the build matches that fill.

R7. Lowest score. In one cycle the tree keeps the smaller valid score. An empty leaf cannot win. An equal score keeps the left input of that match. No ranking is stored. *Block:* `u_sched`. *Check:* `grant_idx` is the leaf that holds the lowest score.

R8. One offer. A record in the ready slot is offered once. `pass_en` consumes it once. `ready_count` is reported to `u_cu` and does not enter the cap. *Block:* `hop_path`, `u_ready`. *Check:* one `pass_en` per write.

R9. Credit safety. A flit leaves only when `credit_count` > 0. The count does not go negative. *Block:* `u_credit`, `u_cu`. *Check:* no `pass_en` while the count is 0.

R10. Credit return. The credit taken by a send returns after $2D_{\text{phy}}$ cycles. The flit itself is `out_valid` at D_{phy}. *Block:* `u_credit`, `u_phy`. *Check:* with $D_{\text{phy}} = 4$ the return is 8 cycles after `pass_en`.

R11. Tile completion. The admitted count drops when `u_ready` consumes that tile's own 16th GEMM flit. That is the cycle the flit enters the PHY. An other-stack consume does not pulse `square_finished`. *Block:* `u_ready`, `u_release`. *Check:* the pulse is on that consume, and the admitted count falls by one.

R12. Delivery. The payload leaves unchanged after $D_{\text{phy}} = 4$ cycles. `u_unpack` hands out its 32 bytes with the tile and the panel. *Block:* `u_phy`, `u_unpack`. *Check:* the byte stream matches the accepted bytes, with the tile and the panel.

6.2 Control unit

`hop_path` wires `u_cu` to the datapath. The control unit decides release and send permission. It does not carry a panel byte, a flit, or a record. It does not scan the ready slots. `u_cu` has no policy pin.

- `release_en`, W, `admit_limit`, and `send_en` go from `u_cu` to the datapath.
- `credit_count`, `ready_count`, and `grant_valid` come back to `u_cu`.
- `grant_idx` stays on the datapath and selects the record.

Output tile k has release time $T_k = T_0 + k \cdot C_{\text{tile}}$. Here $C_{\text{tile}} = 16$ and $T_0 = 0$, so $T_k = 16k$. The deadline stored later is $T_k + 40$. `release_en` asks for the next output tile once the cycle has reached $T_k - C_{\text{tile}}$. `send_en` is high only when `credit_count` is above zero and `grant_valid` is high.

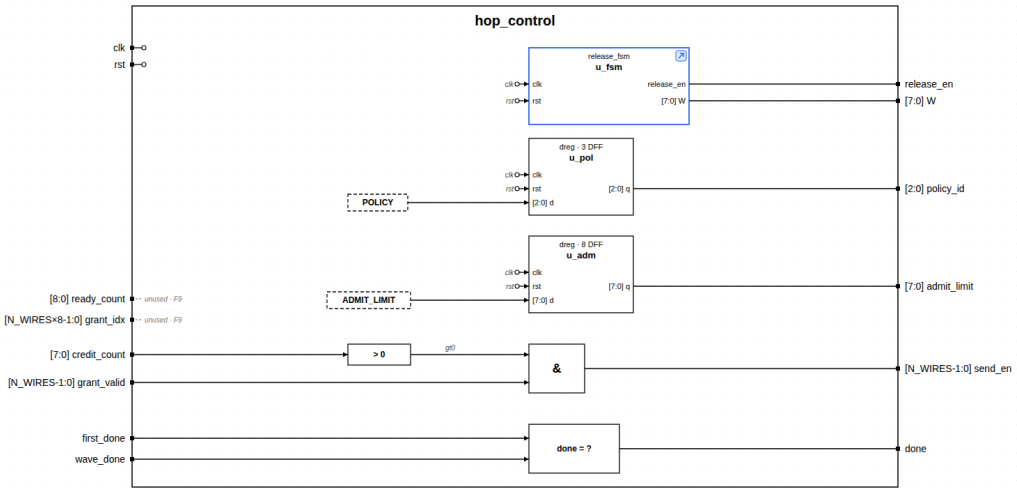


Figure 2: Editor sheet of `hop_control`. `u_fsm` drives `release_en` and `W`. `send_en` is high when the credit count is above zero and `grant_valid` is high.

6.3 Processing unit

The processing unit holds the bytes and the records. The blocks, in order, are `u_release`, `u_packer`, `u_ready`, `u_score`, `u_sched`, `u_credit`, `u_phy`, and `u_unpack`.

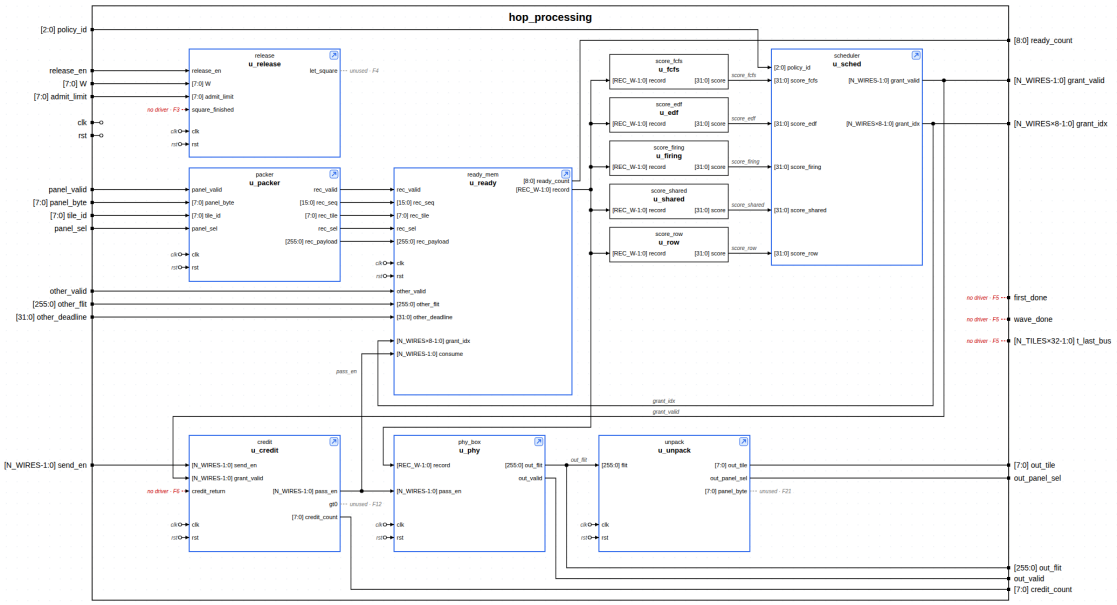


Figure 3: Editor sheet of `hop_processing`. The five score modules sit between `u_ready` and `u_sched`. A blue frame opens the next sheet.

`u_release`. The release block decides when the next output tile may enter the packer. It counts how many output tiles are admitted. Byte acceptance and admission are separate.

- W is the live length and the first input of the cap.
- `w_hold` is the wave length.
- The cap is $\min(W, \texttt{admit_limit})$.
- `let_square` stays high while that output tile is packed. `pack_done` clears it.
- `square_finished` lowers the admitted count by one.

`u_packer`. The packer takes one byte when `let_square` and the panel valid bit are both high. Every 32 accepted bytes become one 256-bit flit. `pack_done` pulses on the 16th flit of the output tile. The output tile stays admitted.

`u_ready`. Ready stores one 336-bit record. The GEMM deadline is $D_k = 16 \cdot \texttt{tile_id} + 40$. An other flit uses the deadline it brings. From the high bit, the record holds valid, the GEMM tag, the wave id, `consumers`, `flits_left`, D_k, `seq`, `panel_sel`, `tile_id`, and the payload. Those widths sum to 336 bits. `score_rec` and the word sent to the PHY are that same record.

`u_score`. The block reads the 336-bit record and packs one 48-bit score. The five fills are the list in Proposed policies. A smaller score wins.

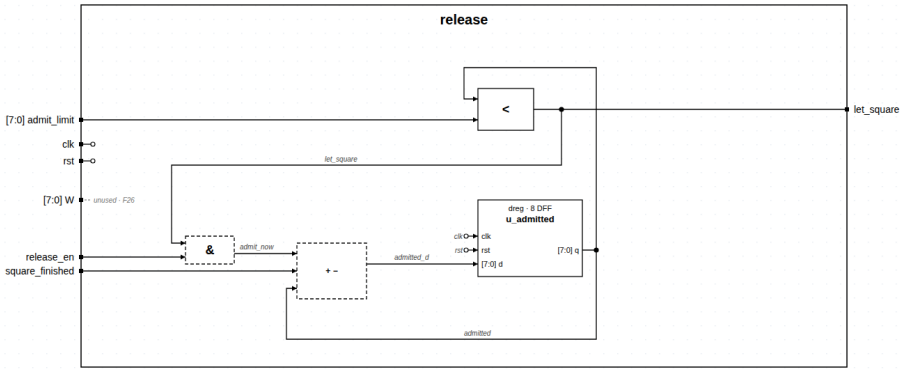


Figure 4: Editor sheet of `release`. `u_admitted` holds the admitted count. `let_square` opens the byte lane.

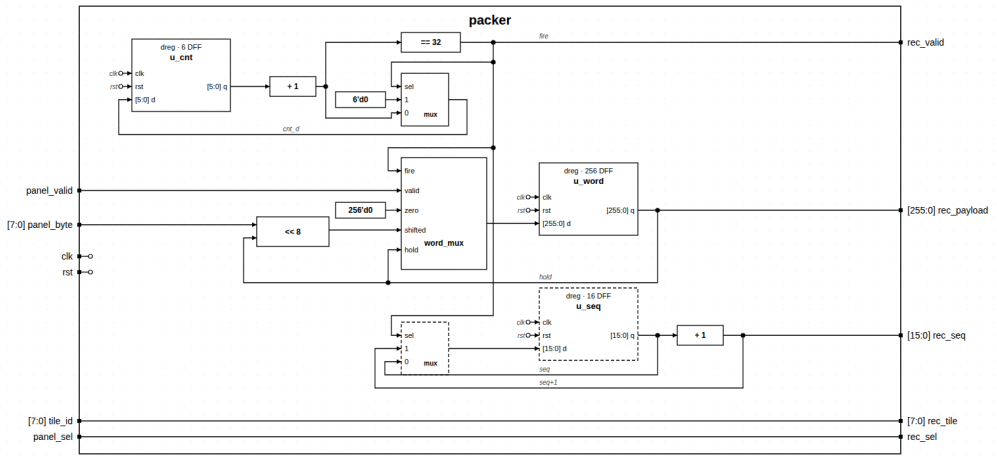


Figure 5: Editor sheet of `packer`. Panel bytes enter on the left. A finished flit leaves toward ready.

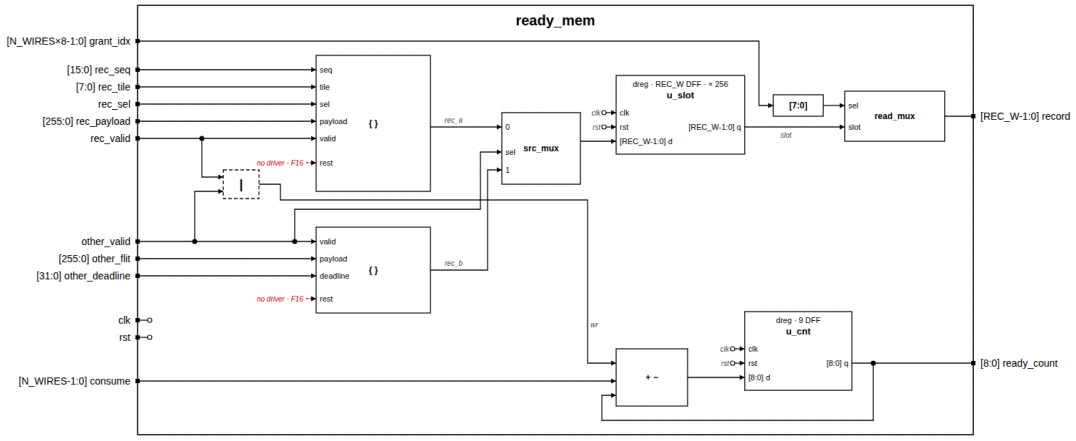


Figure 6: Editor sheet of `ready_mem`. One slot holds the record that the score modules and the PHY both read.

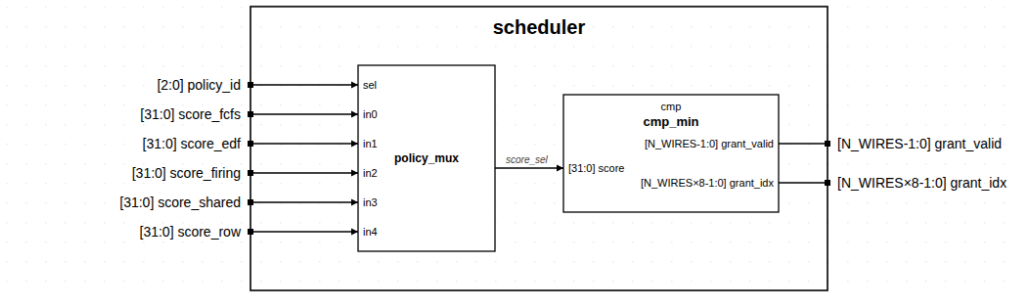


Figure 7: Editor sheet of `scheduler`. `policy_mux` selects one of the five scores. `cmp_min` returns the grant.

`u_sched.` The tree has four leaves and three matches. It contains no register. Each match keeps the smaller valid score and the slot index that came with it. `grant_score` stops at the tree. The PHY receives the record.

`u_credit.` The counter starts at $K = 16$. `pass_en` is high when `send_en`, `grant_valid`, and `credit_count` > 0 are all true. `credit_count` falls by one on `pass_en`. It rises by one on the credit return. The return is $2D_{\text{phy}}$ cycles after the send.

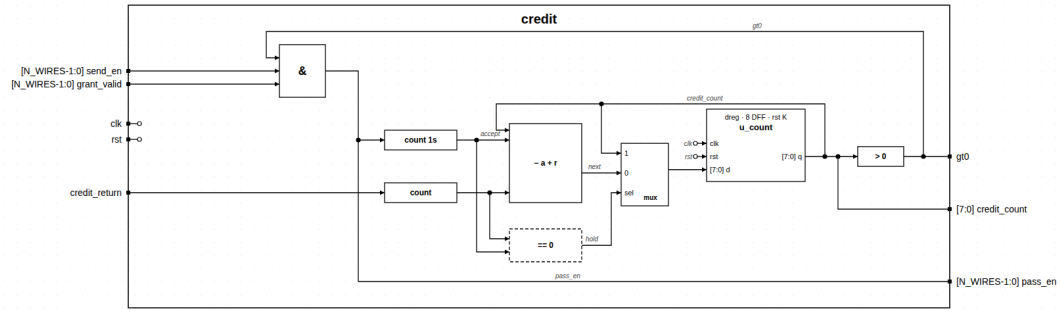


Figure 8: Editor sheet of `credit`. `pass_en` spends one credit. `credit_return` adds it back.

`u_phy.` The PHY is one lane of $D_{\text{phy}} = 4$ registers. `pass_en` loads the granted record into the first stage. `out_flit` is the 256-bit payload of the record that has finished the four cycles. `out_valid` is high on that cycle.

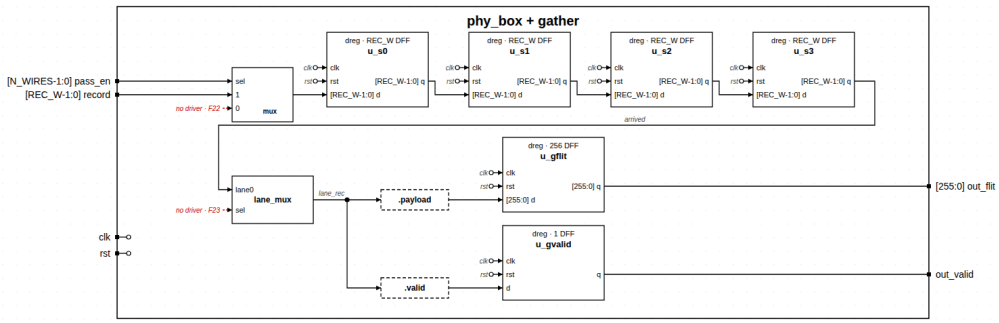


Figure 9: Editor sheet of `phy_box`. `u_s0` through `u_s3` are the four delay registers.

`u_unpack`. Unpack takes the arrived flit. It returns one panel byte at a time, with the tile and the panel. One flit yields 32 bytes.

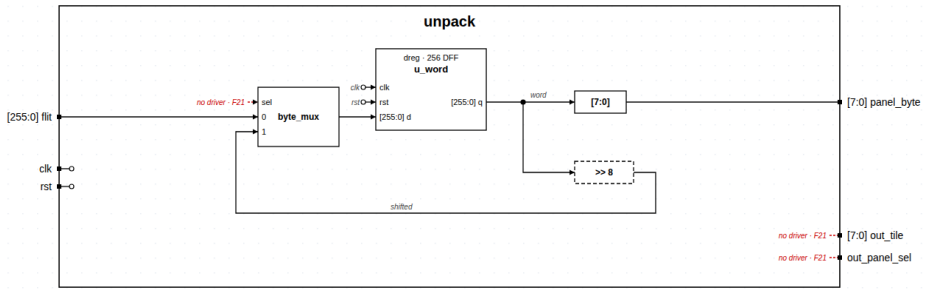


Figure 10: Editor sheet of `unpack`. One arrived flit returns panel bytes, with the tile and the panel.

7 Strategy

The paper is completed in three milestones on the hop drawn above. The first two stay in construction. They widen the physical layer, then the compare tree. The third measures the five score fills on that hop. A later milestone starts only after the one before it matches the base machine.

Milestone 1. More wires in the physical layer. The base machine has one lane. `N_WIRES`, the number of delay lanes in the PHY, is 1. The next build sets `N_WIRES` to 2, and then to 4. Each lane is still a delay of D_{phy} registers. The scheduler emits one grant per lane. The credit block accepts at most that many sends in one cycle, and never more than the free slots. The packer and the ready record stay one flit wide. The score block and unpack stay one flit wide.

Milestone 2. More leaves in the compare tree. The tree now has four leaves. The finished hop still offers one ready slot to that tree. The next build gives the tree one leaf per receive slot, up to $K = 16$. A grant can then choose among the occupied slots. An empty leaf still cannot win. A tie still keeps the lower slot.

Milestone 3. Measure the five fills. The five fills run on the same hop and the same traffic. A rebuild changes the score packing. The rest of the hop stays in place.

Output tile k misses when its last flit arrives after D_k. The miss rate is the number of missed tiles divided by the number of tiles. The lateness of that tile is $L_k = \max(0, t_{\text{last},k} - D_k)$. $t_{\text{last},k}$ is the arrival time of the last flit of output tile k. The useful share is

$$B_{\text{useful}} = \frac{N_{\text{GEMM}}}{N_{\text{GEMM}} + N_{\text{other}}}.$$

N_{GEMM} counts GEMM flits that leave the data die. N_{other} counts other flits that leave on the same hop. A CXL stream is the example.

Area, timing, and power are the cost of the packing. The five builds share one datapath. Only the score block changes with `POLICY`. Each run reports the miss rate, the useful share, and that cost.

`POLICY`	fill	miss rate	B_{useful}	cell area	clock period	power
0	FCFS					
1	EDF					
2	shortest remaining					
3	most consumers					
4	longest remaining					

The cells stay empty until those runs exist.